

FACULDADE QI BRASIL

ENGENHARIA DE SOFTWARE E PADRÕES DE PROJETO

UELINTON JANKE

ATIVIDADE INTEGRADORA:

DA NECESSIDADE AO PROJETO DE SOFTWARE.

Porto Alegre - RS

UELINTON JANKE

UELINTON JANKE

ATIVIDADE INTEGRADORA:

DA NECESSIDADE AO PROJETO DE SOFTWARE.

Projeto de Pesquisa apresentado ao curso de Análise e Desenvolvimento de Sistemas, da Unidade Curricular Engenharia de Software e Padrões de Projeto, a ser utilizado como diretriz da manufatura da Atividade Integradora de Curso

Orientado por: Ph.D. Luciano Zembruzki

Porto Alegre - RS

2026

Resumo Executivo

O presente projeto propõe o desenvolvimento de um Sistema de Reserva de Laboratórios e Equipamentos para uma instituição de ensino fictícia. O cenário identificado envolve a necessidade de organizar o uso dos laboratórios de informática e dos equipamentos utilizados em atividades acadêmicas. Quando as reservas são realizadas por planilhas, mensagens, e-mails ou presencialmente, podem ocorrer conflitos de horários, informações desatualizadas e dificuldades para acompanhar as solicitações.

Como proposta de solução, apresenta-se um sistema web centralizado no qual professores e funcionários poderão consultar os recursos disponíveis, verificar datas e horários, solicitar reservas e acompanhar o andamento dos pedidos. Os administradores serão responsáveis pelo cadastro de usuários e recursos, além da análise das solicitações, podendo aprová-las ou rejeitá-las. O sistema também deverá verificar possíveis conflitos de horários, considerando as solicitações pendentes e as reservas aprovadas.

O principal benefício esperado é melhorar a organização e o controle da utilização dos laboratórios e equipamentos, reduzindo a possibilidade de reservas duplicadas, facilitando o acompanhamento dos pedidos e centralizando as informações em um único ambiente. Dessa forma, a solução busca tornar o processo de reservas mais claro, eficiente e confiável para todos os envolvidos.

1. Contexto e Problema

1.1 Organização

O projeto considera uma instituição de ensino fictícia que disponibiliza laboratórios de informática e equipamentos para a realização de aulas, atividades práticas, trabalhos acadêmicos e outras atividades educacionais.

Atualmente, considera-se que o processo de reserva desses recursos pode ser realizado por diferentes meios, como planilhas, mensagens, e-mails ou solicitações presenciais. A falta de um ambiente centralizado pode dificultar o controle das reservas, a consulta de disponibilidade e o acompanhamento dos pedidos.

Nesse contexto, surge a necessidade de desenvolver uma solução que organize e facilite o gerenciamento dos recursos disponíveis na instituição.

1.2 Usuários envolvidos

O sistema terá três perfis principais de usuários:

- Professores: poderão consultar horários e equipamentos disponíveis, solicitar reservas e acompanhar o andamento de seus pedidos.
- Funcionários: terão acesso às mesmas funcionalidades de consulta, solicitação e acompanhamento de reservas, conforme suas permissões.
- Administradores: serão responsáveis pelo gerenciamento dos usuários e recursos cadastrados, além de analisar as solicitações, podendo aprová-las ou rejeitá-las.

A instituição também será beneficiada indiretamente, pois terá maior controle sobre a utilização dos espaços e equipamentos.

1.3 Necessidade identificada

A principal necessidade é centralizar e organizar o processo de reserva de laboratórios e equipamentos, evitando problemas que podem ocorrer quando as solicitações são controladas manualmente ou por ferramentas diferentes.

Entre os principais problemas que a solução pretende reduzir, destacam-se:

- Conflitos de horários e solicitações duplicadas para o mesmo recurso.
- Dificuldade para consultar a disponibilidade dos laboratórios e equipamentos.

- Falta de padronização no processo de solicitação e aprovação.
- Demora na comunicação sobre a situação das reservas.
- Dificuldade para manter um histórico organizado das solicitações realizadas.

Diante dessas necessidades, propõe-se um sistema web que permita realizar consultas, registrar solicitações, controlar conflitos e acompanhar o processo de aprovação.

1.4 Limites do projeto

O projeto contempla o desenvolvimento de uma solução voltada especificamente ao gerenciamento de reservas de laboratório e equipamentos.

Funcionalidades contempladas:

- Autenticação de usuários.
- Cadastro e gerenciamento de usuários.
- Cadastro e gerenciamento de laboratórios e equipamentos.
- Consulta de recursos e disponibilidade.
- Solicitações de reservas.
- Verificação de conflitos de horários.
- Aprovação e rejeição de solicitações pelos administradores.
- Consulta ao histórico e acompanhamento das solicitações.
- Cancelamento de solicitações ou reservas conforme as regras estabelecidas.

Funcionalidades fora do escopo inicial:

- Desenvolvimento de aplicativo para dispositivos móveis.
- Controle físico de acesso aos laboratórios.
- Integração com catracas ou sistemas de segurança.
- Monitoramento em tempo real da utilização dos equipamentos.
- Integração com sistemas externos da instituição.
- Utilização de inteligência artificial para gerenciamento de reservas.
- Controle financeiro ou cobrança pelo uso dos recursos.

Esses limites foram definidos para manter o projeto concentrado em seu objetivo principal: organizar o processo de reservas e melhorar o controle sobre a

utilização dos recursos acadêmicos. Funcionalidades adicionais poderão ser consideradas em futuras versões, caso sejam identificadas novas necessidades.

2. Requisitos e Premissas

Esta seção apresenta os requisitos funcionais e não funcionais do Sistema de Reserva de Laboratórios e Equipamentos, além das premissas consideradas para o desenvolvimento do projeto. Os requisitos foram definidos com base no cenário proposto, nas necessidades dos usuários e nos objetivos que a solução pretende alcançar.

2.1 Origem das informações

Como o projeto considera uma instituição de ensino fictícia, os requisitos foram elaborados a partir da análise do cenário apresentado e dos problemas que podem ocorrer no gerenciamento de reservas de laboratórios e equipamentos.

Foram consideradas as necessidades dos três principais perfis de usuários: professores, funcionários e administradores. Também foram utilizadas como referência técnicas de levantamento de requisitos comuns na Engenharia de Software, como entrevistas, questionários, observação dos processos e análise de documentos.

Essas técnicas representam possibilidades para uma futura validação dos requisitos com usuários de uma instituição real. Não foram realizadas entrevistas ou pesquisas de campo para elaboração deste projeto.

2.2 Requisitos funcionais

Os requisitos funcionais descrevem as operações e funcionalidades que o sistema deverá oferecer aos usuários.

Código	Requisito funcional	Prioridade
RF01	Permitir que os usuários realizem login utilizando e-mail e senha.	Alta
RF02	Permitir que o administrador cadastre, consulte, altere e desative usuários.	Alta
RF03	Permitir que o administrador cadastre, consulte, altere e desative laboratórios e equipamentos.	Alta

RF04	Permitir que professores e funcionários consultem os recursos cadastrados.	Alta
RF05	Permitir a consulta de disponibilidade dos recursos por data e horário.	Alta
RF06	Permitir que professores e funcionários solicitem reservas, informando recurso, data, horário de início, horário de término e finalidade.	Alta
RF07	Verificar a existência de conflitos com solicitações pendentes ou reservas aprovadas para o mesmo recurso e período.	Alta
RF08	Permitir que o administrador aprove ou rejeite solicitações de reserva.	Alta
RF09	Permitir que os usuários consultem o andamento e o histórico de suas solicitações.	Média
RF10	Permitir que os usuários cancelem suas próprias solicitações ou reservas, respeitando as regras estabelecidas.	Média

As prioridades foram definidas considerando a importância de cada funcionalidade para o funcionamento inicial do sistema. Os requisitos de alta prioridade representam as operações essenciais, enquanto os de média prioridade complementam a solução e podem ser desenvolvidos após as funcionalidades principais.

2.3 Requisitos não funcionais

Os requisitos não funcionais estabelecem critérios de qualidade, segurança, desempenho e utilização que deverão ser considerados durante o desenvolvimento.

Código	Requisito não funcional	Prioridade
RNF01	O sistema deverá exigir autenticação para acessar funcionalidades restritas.	Alta
RNF02	O sistema deverá controlar as permissões de acesso conforme o perfil do usuário.	Alta
RNF03	As operações principais deverão apresentar respostas em até 3 segundos, em condições normais de utilização.	Média
RNF04	A interface deverá ser simples, organizada e de fácil utilização, mesmo para usuários sem conhecimentos técnicos avançados.	Alta
RNF05	O sistema deverá funcionar nos navegadores atuais Google Chrome, Microsoft Edge e Mozilla Firefox.	Média

RNF06	O sistema deverá preservar a integridade das informações, evitando registros duplicados e alterações indevidas.	Alta
RNF07	As senhas deverão ser armazenadas utilizando funções hash apropriadas, como Argon2 ou bcrypt.	Alta
RNF08	A estrutura do sistema deverá facilitar a manutenção e a inclusão de novas funcionalidades.	Média
RNF09	O sistema deverá proteger os dados pessoais dos usuários e restringir seu acesso a pessoas autorizadas, observando os princípios e requisitos aplicáveis da Lei Geral de Proteção de Dados (LGPD).	Alta

O requisito de desempenho estabelece uma meta inicial que deverá ser verificada por meio de testes durante o desenvolvimento. Da mesma forma, os requisitos de segurança e integridade deverão ser considerados desde a implementação das funcionalidades.

2.4 Regras de negócio

As regras de negócio definem as condições que deverão ser respeitadas durante a utilização do sistema.

Código	Regra de negócio
RN01	Somente usuários cadastrados, ativos e autenticados poderão acessar as funcionalidades restritas.
RN02	Apenas administradores poderão cadastrar, alterar ou desativar usuários e recursos.
RN03	Toda solicitação de reserva deverá informar o recurso, a data, o horário de início, o horário de término e a finalidade.
RN04	Não será permitida uma reserva para o mesmo recurso em período conflitante com outra solicitação pendente ou reserva aprovada.
RN05	Uma solicitação somente será considerada confirmada após a aprovação de um administrador.
RN06	Ao rejeitar uma solicitação, o administrador deverá informar o motivo da rejeição.
RN07	Um usuário não poderá cancelar solicitações ou reservas pertencentes a outros usuários.
RN08	Recursos desativados não poderão receber novas solicitações de reserva.

Além disso, a verificação de conflitos deverá ocorrer tanto no momento da criação da solicitação quanto no momento da sua aprovação. Essa medida busca evitar que duas solicitações sejam aprovadas para o mesmo recurso e período.

2.5 Premissas e suposições

Como o cenário é fictício e não foram realizadas pesquisas em uma instituição real, algumas condições foram estabelecidas como premissas para orientar o desenvolvimento do projeto.

As principais premissas são:

- A instituição disponibiliza laboratórios de informática e equipamentos para atividades acadêmicas.
- Professores e funcionários possuem cadastro individual no sistema.
- Os administradores são responsáveis pelo gerenciamento dos usuários e recursos.
- Cada solicitação corresponde a um único recurso por vez, podendo ser um laboratório ou um equipamento.
- As reservas dependem da aprovação de um administrador para serem confirmadas.
- O sistema deverá manter o histórico das solicitações e suas respectivas situações.
- Solicitações rejeitadas ou canceladas não deverão bloquear a disponibilidade do recurso.
- Os usuários terão acesso às funcionalidades de acordo com suas permissões.
- O sistema será desenvolvido inicialmente como uma aplicação web.

Essas premissas permitem estabelecer uma base comum para a modelagem do sistema, evitando que funcionalidades ou comportamentos sejam definidos sem justificativa dentro do cenário proposto.

2.6 Restrições do projeto

As restrições representam condições que limitam o desenvolvimento e a implementação da solução.

Foram estabelecidas as seguintes restrições:

- O sistema será desenvolvido inicialmente para utilização em navegadores web.

- O acesso às funcionalidades dependerá de autenticação e permissões específicas.
- O gerenciamento de usuários e recursos ficará restrito aos administradores.
- Cada solicitação poderá contemplar somente um laboratório ou equipamento.
- Não será possível confirmar reservas com conflitos de data e horário.
- A confirmação de uma reserva dependerá da aprovação administrativa.
- O projeto não contempla integrações com sistemas externos ou equipamentos físicos da instituição.
- O desenvolvimento deverá respeitar o prazo, os recursos e as ferramentas disponíveis para a execução do projeto acadêmico.

2.7 Priorização e validação dos requisitos

A priorização dos requisitos foi realizada considerando o impacto de cada funcionalidade sobre o objetivo principal do sistema.

As prioridades foram classificadas da seguinte maneira:

- Alta: requisito essencial para o funcionamento da solução. Sua ausência compromete uma das operações principais do sistema.
- Média: requisito importante para melhorar a experiência e o gerenciamento, mas que pode ser desenvolvido após as funcionalidades essenciais.
- Baixa: requisito complementar, que poderá ser considerado em futuras versões.

A validação dos requisitos deverá verificar se as funcionalidades implementadas atendem às necessidades previstas. Entre os principais pontos de verificação estão a autenticação dos usuários, o controle de permissões, a consulta de disponibilidade, a prevenção de conflitos e o processo de aprovação ou rejeição das solicitações.

Dessa maneira, os requisitos e as premissas estabelecem a base para as próximas etapas do projeto, orientado a elaboração dos diagramas UML, a definição da arquitetura e o planejamento da implementação do sistema.

3. Alternativas Consideradas

Durante a elaboração do projeto, foram consideradas diferentes alternativas para solucionar os problemas relacionados ao gerenciamento de reservas de laboratórios e equipamentos da instituição de ensino. A análise levou em consideração a facilidade de utilização, a organização das informações, o controle de disponibilidade, a possibilidade de expansão e os custos envolvidos.

3.1 Alternativa 1 – Controle manual por planilhas e documentos

A primeira alternativa consiste em manter o processo de reservas por meio de planilhas eletrônicas, documentos, e-mails e mensagens. Nesse modelo, os usuários enviariam suas solicitações aos responsáveis, que ficariam encarregados de verificar a disponibilidade dos recursos e registrar as reservas manualmente.

Vantagens:

- Baixo custo inicial de implantação.
- Facilidade de utilização de ferramentas já conhecidas.
- Não exige o desenvolvimento de um sistema específico.
- Pode atender instituições com poucos recursos e baixa quantidade de solicitações.

Desvantagens:

- Maior possibilidade de erros no preenchimento e na atualização das informações.
- Dificuldade para identificar conflitos de horários.
- Informações distribuídas em diferentes documentos e canais de comunicação.
- Dependência de verificações manuais pelos responsáveis.
- Dificuldade para manter um histórico centralizado e atualizado.

Essa alternativa apresenta simplicidade e baixo custo inicial, mas pode se tornar pouco eficiente conforme aumenta a quantidade de usuários, recursos e solicitações.

3.2 Alternativa 2 - Utilização de uma ferramenta pronta de gerenciamento de reservas

A segunda alternativa consiste em utilizar uma plataforma já existente para controlar agendamentos e reservas. Nesse modelo, a instituição adotaria uma

ferramenta disponível no mercado, configurando os recursos, os horários e as permissões de acesso conforme suas necessidades.

Vantagens:

- Possibilidade de implantação mais rápida.
- Funcionalidades de agendamento já desenvolvidas.
- Menor necessidade de desenvolver e manter uma solução própria.
- Algumas plataformas oferecem notificações e relatórios.

Desvantagens:

- Possibilidade de custos com licenças ou assinaturas.
- Limitações para adaptar as funcionalidades às regras específicas da instituição.
- Dependência do fornecedor da plataforma.
- Possíveis dificuldades de integração com outros sistemas.
- Nem todas as ferramentas oferecem o controle necessário sobre aprovações, permissões e histórico.

Essa alternativa pode atender parte das necessidades identificadas, mas sua adequação dependeria das funcionalidades disponíveis, das condições comerciais e das possibilidades de personalização.

3.3 Alternativa 3 – Desenvolvimento de um sistema web próprio

A terceira alternativa consiste no desenvolvimento de um sistema web específico para o gerenciamento de reservas de laboratórios e equipamentos. A solução seria construída considerando as necessidades dos professores, funcionários e administradores, permitindo centralizar as informações e estabelecer regras próprias para o processo de reservas.

Entre suas principais funcionalidades estariam a autenticação de usuários, o gerenciamento de recursos, a consulta de disponibilidade, a solicitação de reservas, a verificação de conflitos e a aprovação ou rejeição pelos administradores.

Vantagens:

- Funcionalidades desenvolvidas conforme as necessidades identificadas.

- Maior controle sobre as regras de negócio.
- Centralização das informações e do histórico de reservas.
- Possibilidade de expansão e inclusão de novas funcionalidades.
- Controle de acesso conforme os diferentes perfis de usuários.

Desvantagens:

- Exige tempo para análise, desenvolvimento e testes.
- Necessita de profissionais responsáveis pela manutenção.
- Envolve custos de desenvolvimento, hospedagem e infraestrutura.
- Exige cuidados contínuos com segurança, atualização e integridade das informações.

Apesar de exigir maior esforço inicial, essa alternativa permite desenvolver uma solução alinhada aos requisitos definidos para o projeto.

3.4 Comparação das alternativas

Critério	Controle manual	Ferramenta pronta	Sistema web próprio
Custo inicial	Baixo	Variável	Maior
Implantação	Simples	Geralmente rápida	Exige desenvolvimento
Personalização	Baixa	Depende da ferramenta	Alta
Controle de conflitos	Manual	Depende dos recursos	Conforme as regras definidas
Centralização	Limitada	Geralmente disponível	Disponível
Expansão	Limitada	Depende do fornecedor	Conforme o planejamento
Manutenção	Manual	Responsabilidade do fornecedor, conforme o contrato	Responsabilidade da equipe

3.5 Justificativa da escolha

Após analisar as alternativas, foi escolhido o desenvolvimento de um sistema próprio, pois essa abordagem permite atender de maneira mais específica às necessidades identificadas no cenário proposto.

O controle manual, embora apresente baixo custo inicial, possui limitações relacionadas à organização das informações e à identificação de conflitos. Já a

utilização de uma ferramenta pronta pode reduzir o tempo de implantação, mas depende dos recursos oferecidos pelo fornecedor e das possibilidades de personalização.

O desenvolvimento de uma solução própria permite estabelecer regras específicas para a solicitação, análise e confirmação das reservas, além de oferecer controle sobre os diferentes perfis de usuários e a disponibilidade dos recursos.

A escolha também considera a possibilidade de evolução do sistema. Em versões futuras, poderão ser acrescentadas funcionalidades como notificações automáticas, relatórios gerenciais e visualização de reservas em calendário.

Portanto, a alternativa escolhida apresenta maior alinhamento com os requisitos funcionais e não funcionais definidos para o projeto, oferecendo uma solução que poderá evoluir conforme as necessidades da instituição.

4. Casos de Uso e Modelagem UML

A Linguagem de Modelagem Unificada (UML – Unified Modeling Language) é utilizada para representar diferentes aspectos de um sistema, facilitando a compreensão de sua estrutura, de seu funcionamento e das interações entre os usuários e as funcionalidades disponíveis.

Neste projeto, a UML será utilizada para modelar o Sistema de Reserva de Laboratórios e Equipamentos. Foram elaborados quatro diagramas principais: Diagrama de Casos de Uso, Diagrama de Classes, Diagrama de Sequência e Diagrama de Atividades.

Cada diagrama apresenta uma perspectiva diferente do sistema. Enquanto os casos de uso representam as funcionalidades disponíveis, o diagrama de classes demonstra sua estrutura, e os diagramas de sequência e atividades representam o comportamento e o fluxo das operações.

4.1 Diagrama de Casos de Uso

O Diagrama de Casos de Uso representa as funcionalidades que o sistema deverá oferecer e os atores que interagem com elas. Seu objetivo é demonstrar o que cada perfil de usuário poderá realizar, sem apresentar detalhes internos da implementação.

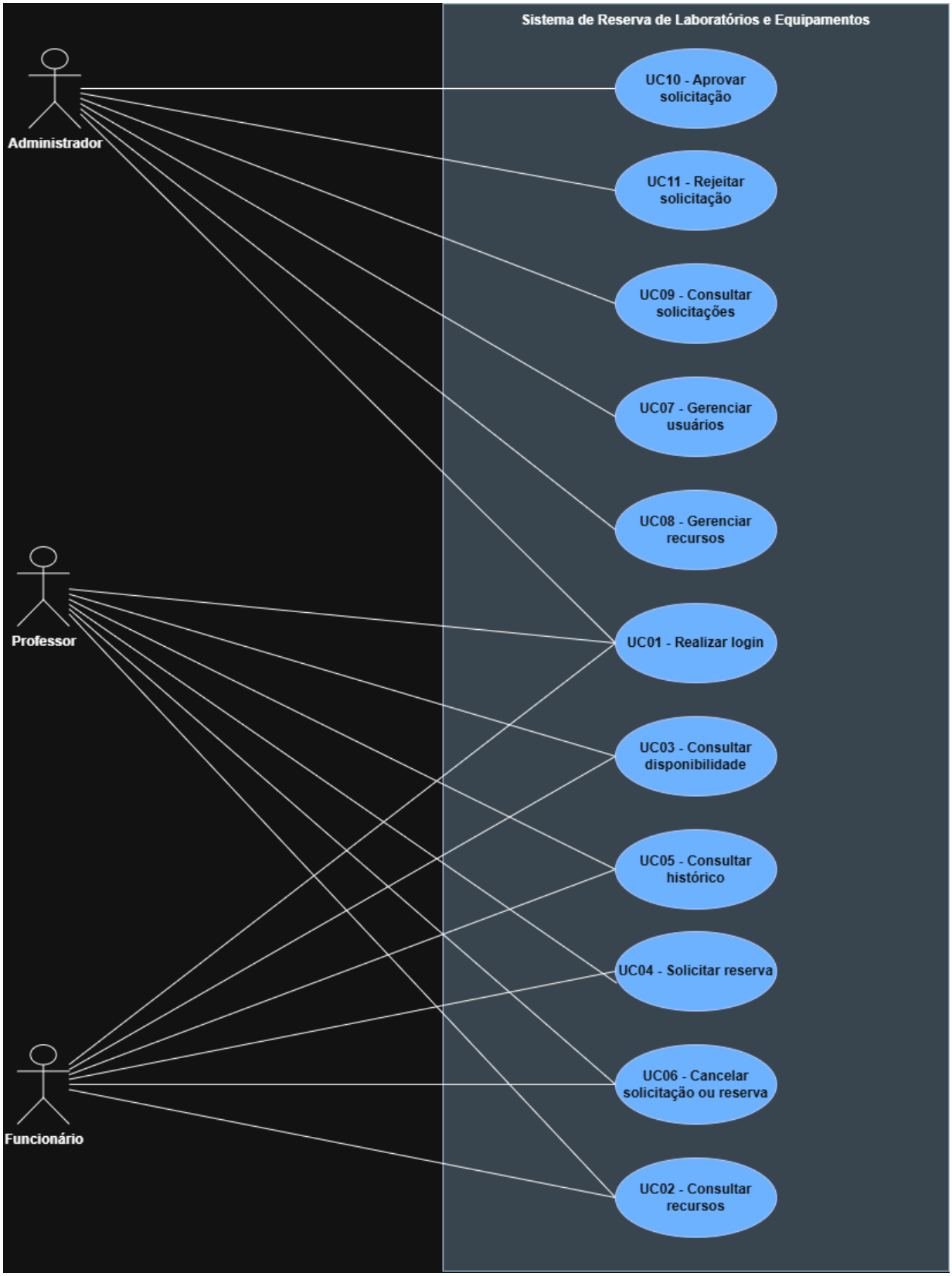


Figura 1 – Diagrama de Casos de Uso. Fonte: Elaborado pelo autor (2026).

Foram definidos três atores principais:

- Professor: poderá realizar login, consultar laboratórios e equipamentos, verificar disponibilidade, solicitar reservas, consultar o histórico e cancelar suas próprias solicitações ou reservas, respeitando as regras estabelecidas.
- Funcionário: terá as mesmas funcionalidades de consulta, solicitação e acompanhamento disponibilizadas ao professor.
- Administrador: será responsável pelo gerenciamento dos usuários e recursos, além de consultar, aprovar e rejeitar solicitações de reserva.

Os casos de uso identificados são:

Código	Caso de uso	Descrição
UC01	Realizar login	Permite que o usuário acesse o sistema utilizando suas credenciais.
UC02	Consultar laboratórios e equipamentos	Permite visualizar os recursos cadastrados.
UC03	Consultar disponibilidade	Permite verificar os recursos disponíveis em determinada data e horário.
UC04	Solicitar reserva	Permite registrar uma solicitação informando recurso, data, horário e finalidade.
UC05	Consultar solicitações e histórico	Permite acompanhar situação e consultar solicitações anteriores.
UC06	Cancelar solicitação ou reserva	Permite cancelar uma solicitação ou reserva própria, conforme as regras do sistema.
UC07	Gerenciar usuários	Permite ao administrador cadastrar, consultar, alterar e desativar recursos.
UC08	Gerenciar laboratórios e equipamentos	Permite ao administrador cadastrar, consultar, alterar e desativar recursos.
UC09	Consultar solicitações de reserva	Permite ao administrador visualizar as solicitações recebidas e seus detalhes.
UC10	Aprovar solicitação	Permite ao administrador aprovar uma solicitação que esteja de acordo com as regras.
UC11	Rejeitar solicitação	Permite ao administrador rejeitar uma solicitação, informando o motivo.

Relacionamento entre atores e funcionalidades

Professores e funcionários compartilham as funcionalidades relacionadas à consulta, solicitação e acompanhamento de reservas. O administrador possui funcionalidades adicionais relacionadas ao gerenciamento e à análise das solicitações.

Para acessar as funcionalidades restritas, os usuários deverão estar autenticados. O administrador também deverá realizar login antes de executar suas operações.

4.2 Especificação dos principais casos de uso

Além da representação gráfica, alguns casos de uso precisam de uma descrição mais detalhada para demonstrar suas condições, etapas e possíveis situações alternativas.

UC04 – Solicitar reserva

- Ator principal: Professor ou Funcionário.
- Objetivo: Registrar uma solicitação de reserva de laboratório ou equipamento.
- Pré-condições: O usuário deverá estar cadastrado, ativo e autenticado no sistema.
- Pós-condição: Uma solicitação deverá ser registrada com o status PENDENTE, caso não exista conflito.

Fluxo principal:

1. O usuário acessa a opção de solicitar reserva.
2. O sistema apresenta os laboratórios e equipamentos cadastrados.
3. O usuário seleciona o recurso desejado.
4. O usuário informa a data, o horário de início, o horário de término e a finalidade.
5. O sistema verifica se os dados foram preenchidos corretamente.
6. O sistema verifica se o recurso está ativo.
7. O sistema verifica se existem solicitações pendentes ou reservas aprovadas para o mesmo recurso e período.
8. Caso não exista conflito, o sistema registra a solicitação.
9. O sistema define o status da solicitação como PENDENTE.
10. O sistema apresenta uma mensagem confirmando o registro.

Fluxos alternativos:

- Se houver campos obrigatórios não preenchidos, o sistema deverá solicitar a correção.
- Se o recurso estiver desativado, o sistema não permitirá a solicitação.

- Se existir conflito de horário, o sistema informará o usuário e não registrará a solicitação naquele período.

UC10 – Aprovar solicitação

- Ator principal: Administrador.
- Objetivo: Aprovar uma solicitação de reserva.
- Pré-condições: O administrador deverá estar autenticado e existir uma solicitação com status PENDENTE.
- Pós-condição: A solicitação deverá apresentar o status APROVADA, caso todas as condições sejam atendidas.

Fluxo principal:

1. O administrador acessa a área de gerenciamento de reservas.
2. O sistema apresenta as solicitações pendentes.
3. O administrador seleciona uma solicitação.
4. O sistema apresenta os dados do usuário, recurso, data, horátio e finalidade.
5. O sistema verifica novamente a existência de conflitos.
6. O administrador confirma a aprovação.
7. O sistema altera o status para APROVADA.
8. O sistema apresenta a confirmação da operação.

Fluxos alternativos:

- Se a solicitação já tiver sido analisada, o sistema deverá informar que não é possível realizar uma nova análise.
- Se for identificado um conflito antes da aprovação, o sistema deverá impedir a confirmação e informar o administrador.

UC11 – Rejeitar solicitação

- Ator principal: Administrador.
- Objetivo: Rejeitar uma solicitação de reserva.
- Pré-condições: O administrador deverá estar autenticado e existir uma solicitação com status PENDENTE.
- Pós-condição: A solicitação deverá apresentar o status REJEITADA e possuir um motivo registrado.

Fluxo principal:

1. O administrador acessa a área de gerenciamento de reservas.
2. O sistema apresenta as solicitações pendentes.
3. O administrador seleciona uma solicitação.
4. O sistema apresenta os detalhes do pedido.
5. O administrador escolhe a opção de rejeitar.
6. O sistema solicita o motivo da rejeição.
7. O administrador informa o motivo.
8. O sistema altera o status para REJEITADA.
9. O sistema confirma a operação.

Fluxos alternativos:

- Se o motivo não for informado, o sistema não permitirá concluir a rejeição.
- Se a solicitação já tiver sido analisada, o sistema deverá impedir uma nova alteração de seu resultado.

4.3 Diagrama de Classes – Estrutura do Sistema

O Diagrama de Classes representa a estrutura estática do sistema, identificando as classes, seus atributos, métodos e relacionamentos.

Para este projeto, foram definidas classes que representam os usuários, os recursos disponíveis, as reservas e o serviço responsável por controlar as regras relacionadas às solicitações.

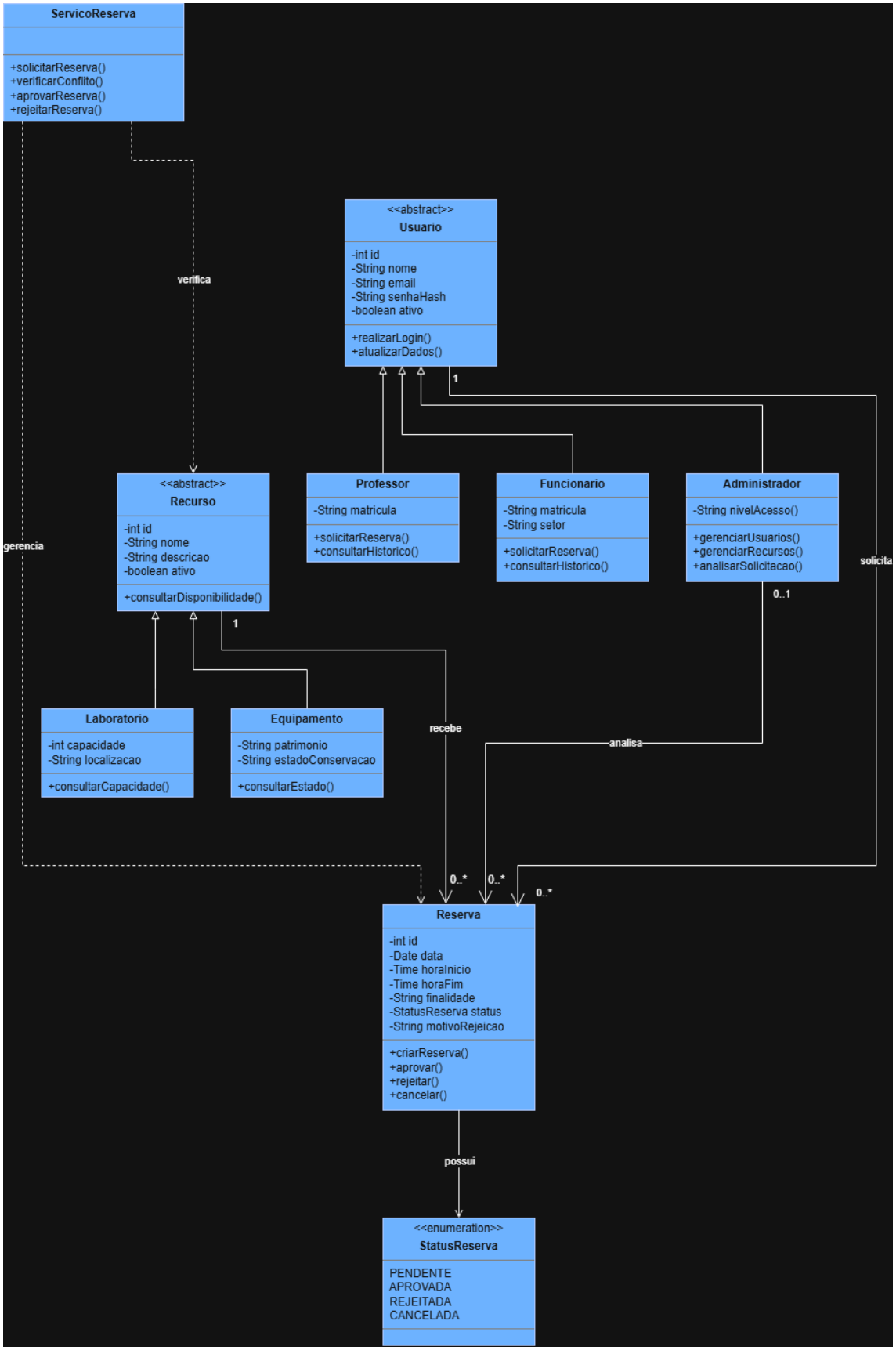


Figura 2 – Diagrama de Classes. Fonte: Elaborado pelo autor (2026).

1. Usuario (classe abstrata)

   Representa os usuários cadastrados no sistema.

   É uma classe abstrata porque representa características comuns aos diferentes perfis de usuários, não sendo utilizada diretamente para criar usuários genéricos.

2. Professor

   Representa o professor que utiliza o sistema.

   Herda os atributos e comportamentos da classe Usuario.

3. Funcionario

   Representa os funcionários que podem solicitar recursos.

   Também herda as características da classe Usuario.

4. Administrador

   Representa o usuário responsável pelo gerenciamento do sistema.

   Herda as características comuns da classe Usuario e possui operações específicas de administração.

5. Recurso (classe abstrata)

   Representa os recursos que podem ser reservados.

   É uma classe abstrata que reúne as características comuns aos laboratórios e equipamentos.

6. Laboratorio

   Representa os laboratórios disponíveis para reserva.

   Herda as características da classe Recurso.

7. Equipamento

   Representa os equipamentos que podem ser solicitados.

   Também herda as características da classe Recurso.

8. Reserva

Representa uma solicitação de reserva realizada por um usuário.

Essa classe concentra as informações e as operações relacionadas ao ciclo de vida de uma reserva.

9. StatusReserva (enumeração)

Define os possíveis estados de uma solicitação:

- PENDENTE
- APROVADA
- REJEITADA
- CANCELADA

A utilização de uma enumeração evita que sejam registrados estados diferentes dos previstos pelo sistema.

10. ServicoReserva

Responsável por coordenar as operações relacionadas às solicitações.

Essa classe centraliza as regras de negócio relacionadas ao processo de reservas, evitando que a lógica de verificação de conflitos fique concentrada na interface do sistema.

Relacionamento entre as classes

Os relacionamentos definidos são:

- Herança: Professor, Funcionario e Administrador herdam de Usuario.
- Herança: Laboratorio e Equipamento herdam de Recurso.
- Associação: um Recurso pode estar relacionado a várias Reservas ao longo do tempo, mas cada Reserva corresponde a um único recurso.
- Associação: um Administrador pode analisar várias Reservas, enquanto uma Reserva poderá ter nenhum ou um administrador responsável pela análise.
- Associação: Reserva possui um StatusReserva.
- Dependência: ServicoReserva utiliza Reserva e Recurso para executar as operações e verificar as regras de negócio.

As classes Usuario e Recurso foram definidas como abstratas porque representam estruturas gerais que serão especializadas por outras classes.

4.4 Diagrama de Sequência - Comportamento do Sistema

O Diagrama de Sequência representa a troca de mensagens entre os participantes durante a execução de uma funcionalidade. Seu objetivo é demonstrar a ordem em que as operações acontecem.

Neste projeto, foi escolhido o processo de solicitação de reserva, pois ele representa uma das principais funcionalidades do sistema.

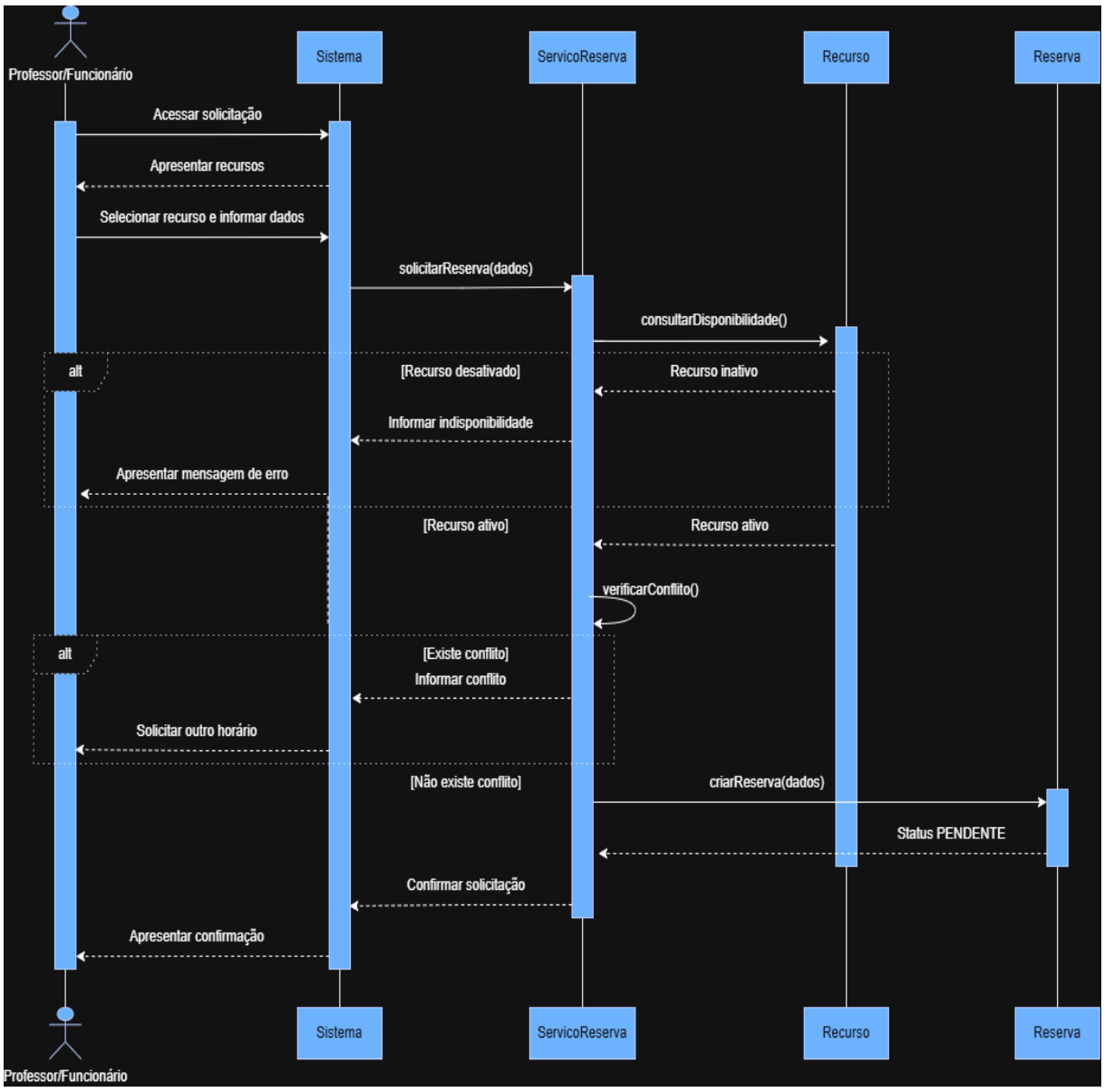


Figura 3 – Diagrama de Sequência. Fonte: Elaborado pelo autor (2026).

Participantes:

- Professor ou Funcionário: usuário que solicita a reserva.

- Sistema: interface responsável por receber as informações e apresentar os resultados.
- ServiceReserva: responsável por executar as regras de negócio.
- Recurso: representa o laboratório ou equipamento selecionado.
- Reserva: representa o registro da solicitação.

Descrição da sequência:

1. O usuário acessa a opção de solicitar reserva.
2. O sistema apresenta os recursos cadastrados.
3. O usuário seleciona um recurso e informa os dados da solicitação.
4. O sistema encaminha as informações para o ServicoReserva.
5. O serviço consulta a situação do recurso.
6. Se o recurso estiver desativado, o sistema apresenta uma mensagem de erro.
7. Se estiver ativo, o serviço executa a verificação de conflitos.
8. A verificação considera solicitações pendentes e reservas aprovadas para o mesmo recurso e período.
9. Se houver conflito, o sistema informa o usuário e não registra a solicitação.
10. Se não houver conflito, o serviço solicita a criação da Reserva.
11. A Reserva é criada com o status PENDENTE.
12. O sistema apresenta uma mensagem confirmando o registro da solicitação.

O diagrama demonstra que a solicitação não é aprovada automaticamente. Mesmo quando não existem conflitos, ela permanece pendente até que um administrador realize a análise.

A consulta dos registros existentes para verificar conflitos é uma responsabilidade interna do serviço de reservas. Os detalhes de banco de dados e persistência não fazem parte desta representação.

4.5 Diagrama de Atividades – Fluxo do Processo

O Diagrama de Atividades representa o fluxo de trabalho do sistema, destacando as ações realizadas, as decisões e os possíveis caminhos durante a execução de um processo.

Para este projeto, o diagrama representa o processo completo de solicitação e análise de uma reserva, desde o acesso do usuário até a aprovação ou rejeição pelo administrador.

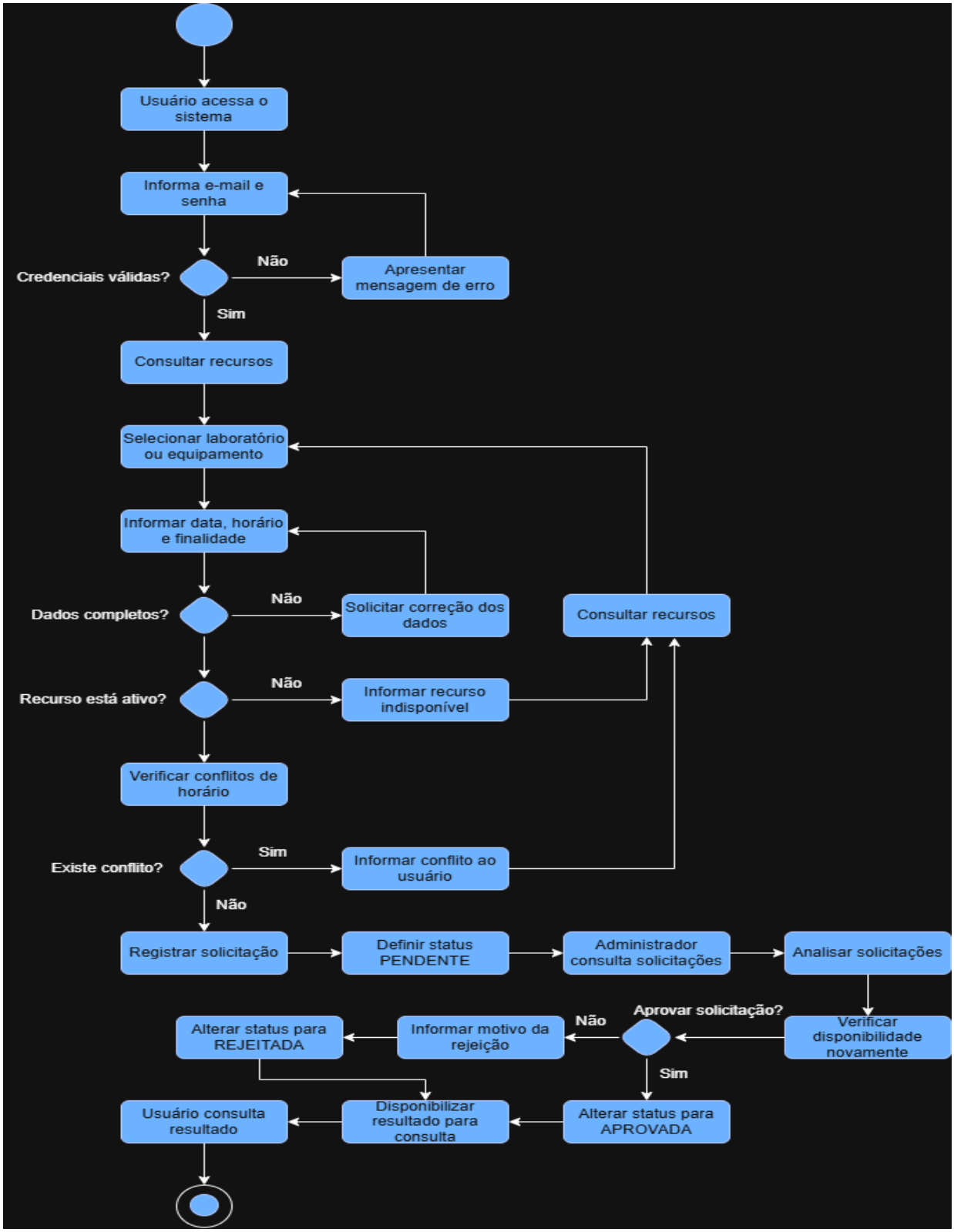


Figura 4 – Diagrama de Atividades. Fonte: Elaborado pelo autor (2026).

Etapas do processo:

1. O usuário acessa o sistema.
2. O sistema solicita as credenciais de acesso.
3. O usuário informa e-mail e senha.
4. O sistema verifica as credenciais.
5. Se os dados estiverem incorretos, apresenta uma mensagem e permite uma nova tentativa.
6. Se estiverem corretos, o usuário acesso o sistema.
7. O usuário consulta os laboratórios e equipamentos disponíveis.
8. O usuário seleciona o recurso desejado.
9. O usuário informa a data, o horário de início, o horário de término e a finalidade.
10. O sistema verifica se todos os campos obrigatórios foram preenchidos.
11. O sistema verifica se o recurso está ativo.
12. O sistema verifica a existência de conflitos.
13. Se houver conflito, o sistema informa o usuário, que poderá escolher outro período ou recurso.
14. Se não houver conflito, o sistema registra a solicitação com o status PENDENTE.
15. O administrador acessa as solicitações pendentes.
16. O administrador analisa os dados da solicitação e verifica novamente a disponibilidade.
17. O administrador decide se aprova ou rejeita o pedido.
18. Se aprovar, o sistema altera o status para APROVADA.
19. Se rejeitar, o administrador informa o motivo e o sistema altera o status para REJEITADA.
20. O usuário poderá consultar o resultado da solicitação.
21. O processo é finalizado.

Decisões representadas no diagrama:

- As credenciais são válidas?
- Os campos obrigatórios foram preenchidos?
- O recurso está ativo?
- Existe conflito de horário?

- O administrador aprovou a solicitação?

Essas decisões permitem representar os diferentes caminhos que o processo poderá seguir, incluindo situações de erro e condições que impendem a continuidade da operação.

O cancelamento não está incluído nesse fluxo principal, pois representa uma operação independente que poderá ser detalhada em outro diagrama de atividades, caso seja necessário.

4.6 Coerência entre os diagramas e os requisitos

Os diagramas foram elaborados considerando os requisitos funcionais e as regras de negócio definidos anteriormente. Cada representação possui uma finalidade específica, mas todas devem apresentar o mesmo comportamento esperado para o sistema.

A relação entre os requisitos e os diagramas pode ser observada a seguir:

Requisitos relacionados	Representação UML
RF01 – Login	Casos de Uso e Diagrama de Atividades
RF02 – Gerenciamento de usuários	Casos de Uso e Diagrama de Classes
RF03 – Gerenciamento de recursos	Casos de Uso e Diagrama de Classes
RF04 e RF05 – Consulta de recursos e disponibilidade	Casos de Uso, Classes e Sequência
RF06 - Solicitação de reserva	Casos de Uso, Classes, Sequência e Atividades
RF07 - Verificação de conflitos	Classes, Sequência e Atividades
RF08 - Aprovação e rejeição	Casos de Uso, Classes e Atividades
RF09 - Histórico e acompanhamento	Casos de Uso e Classes
RF10 – Cancelamento	Casos de Uso e Classes

A modelagem também respeita as regras de negócio relacionadas à autenticação, às permissões dos usuários, à necessidade de aprovação administrativa e à prevenção de conflitos.

É importante destacar que a verificação de conflitos deverá ocorrer tanto no momento do registro da solicitação quando no momento da aprovação. Dessa maneira, o sistema reduz a possibilidade de confirmar reservas incompatíveis para o mesmo recurso e período.

5. Orientação a Objetos

A orientação a objetos é um dos fundamentos utilizados na modelagem do Sistema de Reserva de Laboratórios e Equipamentos. Essa abordagem permite organizar o sistema em classes e objetos, atribuindo responsabilidades específicas a cada elemento e definindo como eles se relacionam para executar as funcionalidades propostas.

No projeto, a modelagem considera as classes Usuario, Professor, Funcionario, Administrador, Recurso, Laboratorio, Equipamento, Reserva e ServicoReserva. Cada uma possui responsabilidades próprias, contribuindo para uma estrutura organizada, reutilizável e de fácil manutenção.

5.1 Responsabilidades das classes

As responsabilidades foram distribuídas conforme a finalidade de cada classe:

- Usuario: representa os usuários cadastrados no sistema. É uma classe abstrata que reúne características comuns, como identificador, nome, e-mail, senha e situação do cadastro.
- Professor: representa o professor que utiliza o sistema para consultar recursos, solicitar reservas e acompanhar seu histórico.
- Funcionario: representa o funcionário que possui permissões para consultar recursos, solicitar reservas e acompanhar suas solicitações.
- Administrador: é responsável pelo gerenciamento de usuários e recursos, além da análise das solicitações de reserva, podendo aprová-las ou rejeitá-las.
- Recurso: é uma classe abstrata que representa os recursos disponíveis para reserva, reunindo informações comuns, como identificador, nome, descrição e situação.
- Laboratorio: representa os laboratórios disponíveis na instituição, armazenando informações específicas, como capacidade e localização.
- Equipamento: representa os equipamentos disponíveis para utilização, contendo informações como número de patrimônio e estado de conservação.
- Reserva: armazena os dados de cada solicitação, incluindo recurso, data, horário, finalidade e situação. Também concentra as operações relacionadas ao ciclo de vida da reserva, como aprovação, rejeição e cancelamento.

- ServicoReserva: concentra a coordenação das regras de negócio relacionadas às reservas, incluindo a verificação de disponibilidade e conflitos de horários, além do processamento das solicitações e decisões administrativas.

Essa divisão permite que cada classe tenha uma finalidade definida, evitando que todas as funcionalidades fiquem concentradas em um único componente.

5.2 Relação entre as classes

As relações entre as classes foram estabelecidas de acordo com as características e necessidades do sistema.

Herança: ocorre quando uma classe especializada herda atributos e comportamentos de uma classe mais genérica. No projeto, Professor, Funcionario e Administrador herdam características de Usuario. Da mesma forma, Laboratorio e Equipamento herdam características de Recurso.

Essa relação permite reutilizar atributos e comportamentos comuns, evitando repetições desnecessárias.

Associação: representa a ligação entre classes que precisam interagir. No sistema, um usuário pode realizar várias reservas, enquanto cada reserva está vinculada a um único usuário. Da mesma forma, um recurso pode estar associado a várias reservas ao longo do tempo, mas cada reserva corresponde a apenas um recurso.

O administrador também se relaciona com as reservas que analisa, sendo responsável por aprovar ou rejeitar as solicitações.

Dependência: ocorre quando uma classe utiliza outra para executar determinada operação. No projeto, ServicoReserva depende de Reserva para processar solicitações e de Recurso para verificar se o recurso está ativo e disponível no período solicitado.

Além disso, a classe Reserva utiliza enumeração StatusReserva, que define os possíveis estados de uma solicitação: PEDENTE, APROVADA, REJEITADA e CANCELADA.

5.3 Coesão

A coesão representa o quanto as responsabilidades de uma classe estão relacionadas entre si. Uma classe apresenta boa coesão quando suas operações e seus atributos estão direcionados a uma finalidade específica.

No Sistema de Reserva de Laboratórios e Equipamentos, busca-se manter uma boa coesão por meio da separação das responsabilidades.

Por exemplo, a classe Reserva concentra informações e operações relacionadas ao processo de reserva, enquanto a classe ServicoReserva coordena as regras de negócio necessárias para validar e processar as solicitações.

Da mesma forma, Laboratorio e Equipamento possuem características específicas de cada tipo de recurso, sem misturar suas informações com as responsabilidades dos usuários ou das reservas.

Essa organização facilita a compreensão do código, reduz a duplicação de responsabilidades e torna mais simples realizar alterações e corrigir problemas.

5.4 Acoplamento

O acoplamento representa o nível de dependência entre as classes de um sistema. O objetivo é manter um baixo acoplamento, permitindo que alterações em uma classe tenham o menor impacto possível sobre as demais.

No projeto, procura-se alcançar esse objetivo utilizando classes abstratas e separando as regras de negócio das demais responsabilidades.

Por exemplo, ServicoReserva coordena as operações relacionadas às solicitações sem precisar concentrar toda a lógica na interface utilizada pelo usuário. Assim, uma alteração na forma como as reservas são apresentadas não precisa modificar diretamente as regras de validação e aprovação.

Outro exemplo é a utilização da classe abstrata Recurso, que permite que o serviço trabalhe com diferentes tipos de recursos, como laboratórios e equipamentos, sem depender exclusivamente de suas implementações específicas.

Com isso, o sistema pode receber novos tipos de recursos ou sofrer alterações em suas funcionalidades com menor necessidade de modificar classes que não estão diretamente relacionadas à mudança.

5.5 Aplicação dos princípios no projeto

A aplicação desses conceitos contribui para uma arquitetura mais organizada e preparada para futuras evoluções. A separação de responsabilidades facilita a manutenção, a herança promove o reaproveitamento de código, a coesão mantém cada classe direcionada à sua finalidade e o baixo acoplamento reduz a dependência entre os componentes.

Portanto, a orientação a objetos permite estruturar o Sistema de Reserva de Laboratórios e Equipamentos de maneira que suas funcionalidades sejam distribuídas entre classes com responsabilidades bem definidas. Essa organização favorece a legibilidade, a reutilização, a manutenção e a possibilidade de adicionar novas funcionalidades sem comprometer toda a estrutura existente.

6. Padrões de Projeto

Os padrões de projeto (Design Patterns) são soluções reutilizáveis para problemas recorrentes no desenvolvimento de software. Sua utilização ajuda a organizar o código, reduzir o acoplamento entre componentes e facilitar a manutenção e a evolução do sistema.

No projeto Sistema de Reserva de Laboratórios e Equipamentos, foram selecionados dois padrões de projeto: Factory Method e Observer. Cada um atende a uma necessidade específica da aplicação e pode contribuir para uma estrutura mais flexível e organizada.

6.1 Factory Method (Padrão Criacional)

O padrão Factory Method tem como objetivo centralizar e organizar a criação de objetos, evitando que diferentes partes do sistema precisem conhecer diretamente os detalhes de instanciação de cada classe.

Aplicação no projeto:

No sistema proposto, existem diferentes tipos de recursos que podem ser cadastrados e disponibilizados para reserva, principalmente laboratórios e equipamentos. Embora ambos sejam recursos, cada um possui características específicas.

Para organizar essa criação, pode ser utilizada uma estrutura baseada em uma classe abstrata CriadorRecurso, responsável por definir um método de criação. Classes especializadas, como CriadorLaboratorio e CriadorEquipamento, implementariam esse método para criar os respectivos objetos.

A estrutura seria organizada da seguinte maneira:

- Recurso: classe abstrata que representa os recursos disponíveis no sistema.
- Laboratorio: classe concreta que representa um laboratório.
- Equipamento: classe concreta que representa um equipamento.
- CriadorRecurso: define método responsável pela criação de recursos.
- CriadorLaboratorio: implementa a criação de objetos da classe Laboratorio.
- CriadorEquipamento: implementa a criação de objetos da classe Equipamento.

Exemplo de funcionamento:

Quando o administrador selecionar a opção de cadastrar um laboratório, o sistema utilizará o CriadorLaboratório para construir o objeto correspondente. Caso selecione o cadastro de um equipamento, será utilizado o CriadorEquipamento.

Dessa forma, o restante da aplicação poderá trabalhar com a abstração Recurso, sem precisar conhecer todos os detalhes de criação de cada tipo específico.

Benefícios para o projeto:

- Organiza o processo de criação de objetos.
- Reduz a dependência direta entre as classes que utilizam os recursos e suas implementações específicas.
- Facilita a inclusão de novos tipos de recursos no futuro.
- Favorece a reutilização e a manutenção do código.

Limitação: esse padrão pode aumentar a quantidade de classes e a complexidade do sistema. Portando, sua utilização é recomendada caso o processo de criação dos recursos apresente variações ou precise ser ampliado. Para um cadastro simples, a instanciação direta pode ser suficiente.

6.2 Observer (Padrão Comportamental)

O padrão Observer permite que um objeto informe automaticamente outros componentes interessados quando ocorre uma alteração em seu estado, sem precisar conhecer diretamente os detalhes de implementação de cada observador.

Aplicação no projeto:

No Sistema de Reserva de Laboratórios e Equipamentos, uma solicitação pode passar por diferentes situações: PENDENTE, APROVADA, REJEITADA ou CANCELADA.

Quando o administrador aprovar ou rejeitar uma solicitação, o estado da reserva será atualizado. Nesse momento, o sistema poderá utilizar o padrão Observer para comunicar a alteração aos componentes interessados.

Para isso, podem ser utilizadas as seguintes estruturas:

- Reserva: representa a solicitação e mantém seu estado atual.
- ObservadorReserva: define uma interface para os componentes interessados nas alterações de uma reserva.
- ServicoNotificacao: recebe as notificações de alteração e coordena sua comunicação.
- NotificacaoEmail: representa um possível componente responsável pelo envio de notificações por e-mail.

Exemplo de funcionamento:

Um professor solicita a utilização de um laboratório e a reserva fica com o status PENDENTE. Posteriormente, o administrador analisa a solicitação e decide aprová-la.

Quando o status for alterado para APROVADA, a reserva poderá notificar os componentes registrados como observadores. Um desses componentes poderá ser responsável por enviar um e-mail ao professor informando que sua solicitação foi aprovada.

O mesmo processo poderá ocorrer quando uma solicitação for rejeitada ou cancelada.

Benefícios para o projeto:

- Permite comunicar alterações de status automaticamente.
- Reduz a dependência entre a classe Reserva e os mecanismos de notificação.
- Facilita a inclusão de novos canais de comunicação, como e-mail ou notificações internas.
- Mantém a lógica de alteração da reserva separada da lógica de comunicação.

Limitação: a utilização desse padrão exige controle dos observadores registrados, evitando notificações duplicadas ou falhas de comunicação. Também é necessário definir como o sistema tratará situações em que o envio de uma notificação não for concluído.

É importante destacar que as notificações automáticas não fazem parte das funcionalidades obrigatórias do escopo inicial. Portanto, a aplicação do Observer representa uma possibilidade de evolução futura, enquanto a consulta do andamento das solicitações permanece como funcionalidade prevista no projeto.

6.3 Justificativa da escolha dos padrões

A escolha do Factory Method e do Observer considera as necessidades atuais e as possibilidades de evolução do Sistema de Reserva de Laboratórios e Equipamentos.

O Factory Method foi selecionado porque o sistema trabalha com diferentes tipos de recursos, como laboratórios e equipamentos, que compartilham características gerais, mas possuem informações específicas. Esse padrão pode organizar a criação desses objetos e facilitar a inclusão de novos tipos de recursos.

O Observer foi selecionado porque o sistema possui diferentes estados para as solicitações de reserva e pode, futuramente, precisar comunicar automaticamente essas alterações aos usuários. Sua utilização permite separar o controle das reservas dos mecanismos de notificação.

Os dois padrões atendem a propósitos diferentes: enquanto o Factory Method está relacionado à criação de objetos, o Observer está relacionado à comunicação entre componentes quando ocorre uma mudança de estado.

7. Metodologia, Qualidade e Riscos

O desenvolvimento do Sistema de Reserva de Laboratórios e Equipamentos exige uma metodologia que permita organizar as atividades, acompanhar a evolução do projeto e garantir que as funcionalidades atendam às necessidades dos usuários. Além disso, é necessário estabelecer critérios de qualidade e identificar possíveis riscos que possam comprometer o funcionamento do sistema.

Para isso, propõe-se a utilização da metodologia ágil Scrum, acompanhada de práticas de controle de qualidade, testes e gerenciamento de riscos.

7.1 Metodologia de desenvolvimento

A metodologia escolhida para o desenvolvimento do projeto é o Scrum, uma abordagem ágil que organiza o trabalho em ciclos curtos chamados Sprints. Cada Sprint possui um período definido e tem como objetivo entregar uma parte funcional do sistema.

Essa metodologia foi escolhida porque permite desenvolver o sistema gradualmente, avaliar os resultados de cada etapa e realizar ajustes conforme as necessidades identificadas durante o desenvolvimento.

No Sistema de Reserva de Laboratórios e Equipamentos, o Scrum possibilita priorizar inicialmente as funcionalidades essenciais, como autenticação, cadastro de recursos e solicitação de reservas, deixando melhorias complementares para etapas posteriores.

A aplicação do Scrum considera três responsabilidades principais:

- Product Owner: responsável por organizar as necessidades do sistema, definir prioridades e manter o Product Backlog atualizado.
- Scrum Master: responsável por apoiar a aplicação do Scrum, facilitar a comunicação da equipe e auxiliar na remoção de impedimentos.
- Developers: responsáveis pelo desenvolvimento das funcionalidades, implementação das regras de negócio, integração dos componentes e realização dos testes.

O trabalho será organizado a partir dos seguintes elementos e eventos:

- Product Backlog: lista organizada de funcionalidades, requisitos e melhorias necessárias para o sistema.

- Sprint Planning: reunião para definir o objetivo da Sprint e selecionar as atividades que serão desenvolvidas.
- Daily Scrum: encontro diário para acompanhar o progresso e identificar impedimentos.
- Sprint Review: apresentação dos resultados desenvolvidos ao final da Sprint.
- Sprint Retrospective: avaliação do processo de trabalho para identificar melhorias na organização e execução das atividades.

Para organizar a implementação, propõe-se um cronograma inicial de dez semanas, dividindo em cinco Sprints de duas semanas cada.

Sprint	Período estimado	Atividades previstas
Sprint 1	Semanas 1 e 2	Preparação do ambiente, estrutura inicial, cadastro de usuários e autenticação.
Sprint 2	Semanas 3 e 4	Cadastro de disponibilidade, solicitação de reservas e validação de conflitos.
Sprint 3	Semanas 5 e 6	Consulta de disponibilidade, solicitação de reservas e validação de conflitos.
Sprint 4	Semanas 7 e 8	Aprovação, rejeição, histórico e cancelamento de solicitações.
Sprint 5	Semanas 9 e 10	Testes gerais, correções, avaliação de qualidade e preparação para implantação.

A divisão em Sprints permite acompanhar o desenvolvimento de maneira incremental, possibilitando identificar problemas antes que eles comprometam etapas posteriores.

7.2 Qualidade do Software

A qualidade do software está relacionada à capacidade de atender aos requisitos definidos, funcionar corretamente, proteger as informações e oferecer uma experiência adequada aos usuários.

No projeto, serão considerados critérios de qualidade relacionados à funcionalidade, confiabilidade, usabilidade, desempenho, segurança, compatibilidade e manutenibilidade.

Funcionalidade: o sistema deverá executar corretamente as operações previstas, como autenticação cadastro de recursos, consulta de disponibilidade e gerenciamento de reservas

Confiabilidade: o sistema deverá manter a consistência das informações, evitando reservas duplicadas, alterações indevidas de status e perda de dados.

Usabilidade: a interface deverá ser simples e intuitiva, permitindo que professores, funcionários e administradores utilizem as funcionalidades sem conhecimentos técnicos avançados.

Desempenho: as principais operações deverão apresentar tempo de resposta de até três segundos em condições normais de utilização, considerando um ambiente de infraestrutura adequado.

Segurança: o sistema deverá proteger os dados dos usuários, controlar permissões de acesso e impedir que pessoas não autorizadas executem operações administrativas.

Compatibilidade: a aplicação deverá funcionar adequadamente nas versões atuais dos principais navegadores, como Google Chrome, Microsoft Edge e Mozilla Firefox.

Manutenibilidade: o código deverá ser organizado em componentes com responsabilidades definidas, facilitando correções, atualizações e inclusão de novas funcionalidades.

Para verificar se os critérios estabelecidos estão sendo atendidos, propõe-se a aplicação de diferentes tipos de testes e controles durante o desenvolvimento.

- Testes unitários: verificar o funcionamento individual de métodos e classes, como a validação de horários e a verificação de conflitos.
- Testes de integração: verificar a comunicação entre os componentes, como a interação entre ServicoReserva, Reserva e Recurso.
- Testes funcionais: verificar se as funcionalidades atendem aos requisitos definidos, desde o login até a aprovação ou rejeição de uma reserva.
- Testes de segurança: verificar o controle de acesso, as permissões dos perfis e a proteção das senhas e informações pessoais.
- Testes de desempenho: avaliar o tempo de resposta das operações principais em condições representativas de utilização.
- Testes de usabilidade: avaliar se os usuários conseguem compreender a interface e realizar as operações sem dificuldades relevantes.

Uma funcionalidade será considerada concluída quando atender aos requisitos estabelecidos e passar pelos testes correspondentes.

Entre os principais critérios de aceitação do sistema, destacam-se:

1. Permitir que usuários cadastrados e ativos realizem autenticação.
2. Restringir as operações administrativas ao perfil de administrador.
3. Permitir o cadastro, a consulta, a alteração e a desativação de recursos.
4. Permitir a consulta de laboratórios e equipamentos disponíveis.
5. Registrar solicitações com recurso, data, horário e finalidade.
6. Impedir solicitações com conflitos de horário para o mesmo recurso.
7. Registrar novas solicitações com status PENDENTE.
8. Permitir que o administrador aprove ou rejeite solicitações.
9. Exigir justificativa para a rejeição de uma solicitação.
10. Permitir que os usuários consultem suas solicitações e respectivos status.
11. Permitir o cancelamento conforme as regras de negócio estabelecidas.
12. Manter a integridade das informações durante as operações.

Um controle especialmente importante será a verificação de disponibilidade tanto no momento da solicitação quanto no momento da aprovação. Isso evita que uma reserva seja aprovada com base em uma disponibilidade que já tenha sido alterada.

7.3 Gerenciamento de riscos

O gerenciamento de riscos consiste em identificar situações que podem prejudicar o desenvolvimento ou o funcionamento do sistema, avaliar suas possíveis consequências e estabelecer medidas para reduzir sua ocorrência ou seus impactos.

No projeto, foram identificados riscos relacionados à integridade das reservas, segurança das informações, disponibilidade dos dados, utilização do sistema e cumprimento do cronograma.

Risco identificado	Possível consequência	Controle ou medida preventiva
Conflitos entre reservas	Dois usuários solicitarem o mesmo recurso no mesmo período.	Verificação de disponibilidade no momento da solicitação e da aprovação, com controle de integridade no banco de dados.

Acesso não autorizado	Usuários executarem operações que não correspondem às suas permissões.	Autenticação, controle de acesso por perfil e validação das permissões no servidor.
Perda de informações	Dados de usuários, recursos e reservas serem perdidos.	Realização de backups periódicos e testes de restauração.
Falhas na aprovação de reservas.	Solicitações permanecerem em estados incorretos ou serem aprovadas indevidamente.	Validação das transições de status e testes dos processos de aprovação e rejeição.
Dificuldade de utilização	Usuários apresentarem dificuldades para solicitar ou acompanhar reservas.	Desenvolvimento de interface intuitiva, testes de usabilidade e ajustes com base no feedback.
Atrasos no desenvolvimento	Funcionalidades não serem concluídas dentro do cronograma estimado.	Priorização dos requisitos, acompanhamento das Sprints e revisão das atividades planejadas.
Falhas de desempenho	Lentidão durante consultas e operações de reserva.	Testes de desempenho, otimização de consultas e acompanhamento do tempo de resposta.

Além das medidas preventivas, propõe-se acompanhar os riscos durante todo o desenvolvimento, pois novos problemas podem surgir conforme as funcionalidades forem implementadas.

Para isso, serão adotadas as seguintes estratégias:

- Acompanhamento contínuo: revisar os riscos durante as Sprints e identificar novos problemas.
- Priorização: dar maior atenção aos riscos que podem comprometer a segurança, a integridade das reservas e o funcionamento das operações principais.
- Registro de ocorrências: documentar problemas identificados, suas causas e as medidas adotadas.
- Testes frequentes: verificar as funcionalidades ao longo do desenvolvimento, evitando concentrar todos os testes na etapa final.
- Revisão das entregas: avaliar os resultados de cada Sprint e corrigir falhas antes de avançar para funcionalidades mais complexas.

8. Plano Futuro e Conclusão

O desenvolvimento do Sistema de Reserva de Laboratório e Equipamentos foi estruturado a partir da identificação de um problema, levantamento de requisitos, análise de alternativas, modelagem dos casos de uso, definição da arquitetura orientada a objetos e seleção de padrões de projeto e metodologia de desenvolvimento.

Com base nessas etapas, é possível estabelecer um plano futuro para orientar a implementação, identificar as limitações da proposta e definir as validações necessárias antes de disponibilizar o sistema para utilização.

8.1 Plano futuro e próximos passos

A próxima etapa do projeto consiste em transformar a modelagem e os requisitos definidos em uma aplicação funcional. Para isso, propõe-se uma sequência de atividades que permita desenvolver o sistema de maneira organizada e incremental.

Inicialmente, será necessário preparar o ambiente, configurar as ferramentas, criar o repositório de versionamento e organizar a estrutura inicial da aplicação.

Desenvolver a estrutura do banco de dados para armazenar usuários, laboratórios, equipamentos e reservas. Também deverão ser definidas as relações entre as entidades e as restrições necessárias para manter a integridade das informações.

Implementar o acesso por e-mail e senha, o controle de permissões e as funcionalidades administrativas de cadastro, consulta, alteração e desativação de usuários.

Implementar as funcionalidades para cadastrar, consultar, alterar e desativar laboratórios e equipamentos, permitindo que o administrador mantenha as informações atualizadas.

Desenvolver a consulta de disponibilidade e o processo de solicitação de reservas. Essa etapa deverá contemplar a validação de datas, horários, situação dos recursos e conflitos com outras solicitações pendentes ou reservas aprovadas.

Implementar as funcionalidades de consulta, aprovação, rejeição e cancelamento das solicitações, respeitando as regras de negócio estabelecidas.

Realizar testes unitários, de integração, funcionais, de segurança, desempenho e usabilidade. As falhas identificadas deverão ser corrigidas antes da implantação.

Após a validação das funcionalidades, preparar o ambiente de produção, realizar a implantação, configurar os mecanismos de backup e estabelecer procedimentos de manutenção e atualização.

Para uma possível execução do projeto, considera-se uma equipe organizada de acordo com as responsabilidades do Scrum:

- Product Owner: organiza os requisitos, representa as necessidades dos usuários e define as prioridades.
- Scrum Master: acompanha a aplicação da metodologia, facilita a comunicação e auxilia na resolução de impedimentos.
- Developers: desenvolvem a interface, a lógica de negócio, a integração com o banco de dados e os testes.

Em uma equipe pequena, uma mesma pessoa poderá acumular mais de uma responsabilidade, desde que as atribuições estejam claramente definidas.

Como proposta tecnológica inicial, poderão ser utilizadas as seguintes ferramentas:

Ferramenta ou tecnologia	Finalidade
Java e Spring Boot	Desenvolvimento das regras de negócio e da API.
PostgreSQL	Armazenamento e gerenciamento dos dados.
HTML, CSS e JavaScript	Desenvolvimento da interface web.
Git e GitHub	Versionamento e organização do código-fonte.
IntelliJ IDEA ou Visual Studio Code	Ambiente de desenvolvimento.
JUnit	Realização de testes automatizados.
Figma	Planejamento e prototipação das interfaces.

A escolha definitiva dessas tecnologias deverá considerar a experiência da equipe, os recursos disponíveis, os custos de infraestrutura e as necessidades de manutenção.

8.2 Limitações do projeto

Apesar de apresentar uma proposta estruturada, o projeto possui limitações que precisam ser consideradas antes de sua implementação.

A primeira limitação está relacionada à natureza do cenário utilizado. A instituição de ensino apresentada é fictícia, portanto, os requisitos foram definidos com base em uma situação hipotética, e não em um levantamento realizado diretamente com uma instituição real.

Outra limitação é que o projeto contempla inicialmente apenas um sistema web. Não estão previstas, nesta primeira versão, funcionalidades específicas para aplicativos móveis, integração com catracas, controle físico de acesso aos laboratórios ou monitoramento em tempo real dos equipamentos.

Também não foram previstas integrações com sistemas acadêmicos externos, mecanismos de inteligência artificial, pagamentos ou notificações automáticas. Essas funcionalidades poderão ser avaliadas em versões futuras, caso sejam identificadas necessidades que justifiquem sua implementação.

Além disso, a proposta ainda não possui uma implementação funcional submetida a testes reais. Dessa forma, os critérios de desempenho, segurança, usabilidade e confiabilidade foram estabelecidos como objetivos a serem verificados, não como resultados já comprovados.

Por fim, o cronograma de dez semanas representa uma estimativa de planejamento acadêmico. O prazo efetivo dependerá da quantidade de desenvolvedores, da experiência da equipe, da disponibilidade de recursos e da complexidade encontrada durante a implementação.

8.3 Validações necessárias

Antes de considerar o sistema concluído e preparado para utilização, será necessário validar os requisitos, os processos, a segurança e o funcionamento das funcionalidades implementadas.

As principais validações propostas são:

Validação dos requisitos: verificar se os requisitos funcionais e não funcionais representam corretamente as necessidades dos usuários.

Essa etapa deverá envolver, preferencialmente, professores, funcionários e administradores de uma instituição real, permitindo identificar necessidades não contempladas inicialmente e possíveis ajustes nas regras de negócio.

Validação dos diagramas e da modelagem: conferir se os diagramas UML representam corretamente o funcionamento esperado do sistema, verificando:

- Se os atores possuem permissões adequadas.
- Se os casos de uso correspondem aos requisitos funcionais.
- Se as classes apresentam responsabilidades coerentes.
- Se as relações entre as classes estão corretamente definidas.
- Se os fluxos de sequência e atividades representam os processos de solicitação e aprovação.

Essa validação é importante para evitar inconsistências entre a documentação e a implementação.

Validação das regras de negócio: testar especialmente as regras relacionadas à disponibilidade e ao gerenciamento das reservas.

Alguns cenários importantes são:

- Impedir que um mesmo recurso seja reservado em períodos conflitantes.
- Permitir solicitações para períodos disponíveis.
- Considerar solicitações pendentes e reservas aprovadas na verificação de conflitos.
- Não bloquear a disponibilidade por solicitações rejeitadas ou canceladas.
- Impedir novas solicitações para recursos desativados.
- Exigir justificativa para rejeitar uma solicitação
- Garantir que somente administradores possa aprovar ou rejeitar reservas.

Também será necessário verificar o comportamento do sistema quando dois usuários tentarem solicitar o mesmo recurso para períodos coincidentes simultaneamente.

Validação da segurança: verificar se a autenticação, o armazenamento de senhas de permissões e a proteção dos dados pessoais estão funcionando corretamente.

Também deverá ser avaliada a adequação do sistema à Lei Geral de Proteção de Dados Pessoais (LGPD), considerando o tratamento, o armazenamento e o acesso às informações dos usuários.

Validação da experiência do usuário: realizar testes de usabilidade para verificar se professores, funcionários e administradores conseguem executar suas tarefas de maneira simples e intuitiva.

A avaliação deverá considerar a clareza das informações, a organização das telas, a facilidade de navegação, a acessibilidade e a compreensão das mensagens apresentadas pelo sistema.

Validação do desempenho e da confiabilidade: verificar se as operações atendem ao tempo de resposta estabelecido de até três segundos em condições normais de utilização.

Também deverão ser realizados testes de backup e restauração, além de verificações sobre a consistência dos dados após operações de cadastro, alteração, aprovação, rejeição e cancelamento.

8.4 Possibilidades de evolução

Após a implementação e validação da primeira versão, o sistema poderá receber novas funcionalidades para ampliar sua utilidade.

Entre as possibilidades de evolução, destacam-se:

- Notificações automáticas: envio de avisos por e-mail ou notificações internas sobre aprovações, rejeições e cancelamentos.
- Relatórios administrativos: apresentação de informações sobre utilização dos laboratórios, frequência de reservas e recursos mais solicitados.
- Calendário de reservas: visualização das reservas em formato de calendário, facilitando o planejamento das atividades acadêmicas.
- Aplicativo móvel: disponibilização das principais funcionalidades em dispositivos móveis.
- Histórico de utilização: registro das reservas efetivamente realizadas, permitindo acompanhar a utilização dos recursos.

- Integração com sistemas acadêmicos: possibilidade de integração com plataformas institucionais para facilitar o gerenciamento de usuários e atividades.

Essas melhorias deverão ser avaliadas conforme as necessidades identificadas, os recursos disponíveis e as prioridades definidas para cada nova versão.

9. Referências Bibliográficas

GAMMA, Erich; HELM, Richard; JOHMSON, Ralph; VLISSIDES, John. **Design Patterns: Elements of Reusable Object-Oriented Software.** Boston: Addison-Wesley Professional, 1994.

OBJECT MANAGEMENT GROUP (OMG). **OMG Unified Modeling Language (OMG UML)**, Version 2.5.1. 2017. Disponível em: https://www.omg.org/spec/UML/. Acesso em: 31 out. 2026.

PRESSMANN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game**. 2020. Disponível em: https://scrumguides.org/scrum-guide. Acesso em: 3 nov. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: Portal do Governo Federal – LGPD. Acesso em: 7 nov. 2026.

Ferramentas utilizadas

Para apoiar a elaboração e a organização do relatório, foram utilizadas as seguintes ferramentas:

- **ChatGPT (OpenAI)**: apoio à organização das ideias, estruturação dos requisitos, explicação dos conceitos de Engenharia de Software e revisão da coerência entre as seções.
- **Draw.io**: recurso utilizado para representar visualmente os diagramas UML e os fluxos do sistema.

- **Navegador web**: consulta à documentação técnica, às referências bibliográficas e às fontes oficiais relacionadas aos assuntos abordados.